

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 20%

QUESTIONS: 05

Total Marks:100

Annexure A

Industry Problem Solving

Code of Conduct:

I ADHITHYA.S – 192412352 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1, A retail company wants to recommend products to customers based on past purchases. Design a solution using K-Nearest Neighbour or Case-Based Learning. Explain how similarity is measured, how recommendations are generated, and how the system adapts to new customer data.

2. A manufacturing industry needs to predict machine output based on sensor readings that vary over time. Propose a solution using Locally Weighted Regression. Explain how local models improve prediction accuracy and how the system handles noisy or incomplete data.

3. A healthcare company aims to classify patients into risk categories using historical medical data. Suggest a model using KNN or RBF networks. Explain how the algorithm handles large datasets, feature selection, and accuracy improvement in real-world scenarios.

4. An e-commerce platform wants to detect similar products for dynamic pricing strategies. Design a solution using Case-Based Learning. Discuss how similarity metrics are defined, how past cases are reused, and how the system scales with increasing product data.

5. A logistics company needs to forecast delivery times based on location and traffic patterns. Propose a solution using Radial Basis Function networks or Locally Weighted Regression. Explain how the model captures non-linear relationships and improves prediction reliability under varying conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

CO4 AT2 - Industry Problem Solving

Name: ADHITHYA.S

Register Number: 192412352

Course Code: ITA0606

Course Name: MACHINE LEARNING

1. Product Recommendation System (KNN / Case-Based Learning)

Proposed Design

A hybrid lazy-learning approach: represent each customer as a feature vector built from purchase history — RFM features (Recency, Frequency, Monetary value), category spend distribution, and browsing/purchase sequence. Two complementary techniques apply directly:

- KNN (collaborative filtering): find the K customers most similar to the target customer and recommend items those neighbours purchased that the target hasn't bought yet, ranked by how many/how similar neighbours bought each item.
- Case-Based Learning: treat each past (customer profile → purchased item → outcome) as a stored case; for a new customer query, retrieve the closest matching case(s) and reuse the associated product recommendation, adapting it (e.g., substituting an out-of-stock item with the next most similar) when needed.

Measuring Similarity

- Cosine similarity on sparse purchase vectors (works well for high-dimensional item spaces, since it ignores raw purchase-count magnitude and focuses on purchase pattern overlap).
- Weighted Euclidean distance on engineered RFM/behavioural features, with feature weights reflecting business importance (e.g., recency weighted higher than raw frequency for fast-changing preferences).
- Jaccard similarity on purchased-category sets when comparing customers primarily by which product categories they engage with.

Generating Recommendations

- Retrieve the K nearest customers (or the closest matching case) to the target customer.
- Aggregate the items those neighbours purchased, weighting each item's score by the similarity of the neighbour who bought it (closer neighbours count more).
- Filter out items the target customer already owns, rank remaining items by aggregated weighted score, and present the top-N as recommendations.

Adapting to New Customer Data

- Both KNN and Case-Based Learning are lazy learners — there is no explicit training phase to rerun; new purchase records are simply added to the dataset/case base and are immediately available for future similarity searches.
- As the customer/product base grows, brute-force distance search becomes slow; adopt efficient indexing (KD-tree/Ball-tree for moderate dimensions, or approximate nearest-neighbour structures such as LSH/HNSW for large, high-dimensional catalogues) and periodically re-index.
- Apply recency weighting/decay so older purchases contribute less over time, letting the system track evolving preferences without manual retraining.

- Handle cold-start (new customers with no history) by falling back to popularity-based or content-based recommendations until enough purchase history accumulates for reliable similarity matching.

2. Machine Output Prediction from Sensor Data (Locally Weighted Regression)

Proposed Design

Model machine output as a function of multiple time-varying sensor readings (temperature, vibration, pressure, load, etc.). For each new query (the current sensor reading), Locally Weighted Regression (LWR) fits a fresh local linear model using only the historical readings most similar to the current operating condition, weighted by a Gaussian kernel on their distance (in sensor-feature space, and optionally in time) from the query point.

How Local Models Improve Prediction Accuracy

- Machine behaviour is rarely globally linear — the relationship between sensor readings and output often differs across operating regimes (e.g., startup, steady-state, high-load), which a single global regression line cannot capture.
- LWR effectively performs piecewise, regime-aware modelling: because a new local model is fit around every query, it automatically adapts to whichever regime the current sensor readings fall into, rather than forcing one compromise fit across all regimes.
- This typically yields much lower prediction error than global linear regression whenever the true input-output relationship is non-linear or regime-dependent — the same effect demonstrated when comparing LWR to standard linear regression on a curved function.

Handling Noisy or Incomplete Sensor Data

- Noise robustness: the Gaussian kernel naturally down-weights points that are far from the query, and combining this with a robust loss (e.g., Huber loss instead of squared error) further reduces the influence of individual noisy/outlier sensor readings on the local fit.
- Bandwidth tuning: a wider kernel bandwidth (τ) averages over more historical points, smoothing out sensor noise at the cost of some local precision — this can be tuned based on how noisy a given sensor channel is.
- Missing values: use partial-distance calculations (compute similarity using only the sensor channels that are present) or nearest-neighbour imputation to fill missing readings before applying LWR, so a temporarily malfunctioning sensor doesn't block prediction.
- Concept drift: maintain a sliding window of recent historical data (rather than the entire history) so the local models stay relevant as equipment wears or is recalibrated over time, and so per-query computation stays manageable for near-real-time prediction.

3. Patient Risk Classification (KNN or RBF Network)

Proposed Design

Approach	How it works here	When to prefer it
KNN	Find the K historical patients most clinically similar to the new patient (by age, vitals, lab values, comorbidities) and assign the majority risk category among them.	Best when interpretability matters — clinicians can inspect the actual similar patient cases behind a prediction.
RBF Network	Gaussian units centered on representative patient profiles (e.g., found via clustering)	Best when a smoother, probabilistic risk score (not just a

Approach	How it works here	When to prefer it
	combine to produce a smooth risk score/probability rather than a hard neighbour vote.	category) is needed, and the underlying risk surface is genuinely continuous/non-linear.

Handling Large Datasets

- Brute-force KNN search over a large patient population is computationally expensive (compares against every record); use spatial indexing (KD-tree/Ball-tree) for moderate feature counts, or approximate nearest-neighbour methods for very large/high-dimensional data.
- For RBF networks, keep the number of hidden (Gaussian) units bounded by using representative cluster centers (e.g., via K-Means over the patient population) rather than one unit per training record — this keeps network size, training cost, and inference latency manageable regardless of how large the dataset grows.
- Dimensionality reduction (e.g., PCA) before distance computation both speeds up search and mitigates the curse of dimensionality that otherwise degrades distance-based methods on very high-dimensional clinical records.

Feature Selection

- Combine clinical domain knowledge (known risk factors: age, blood pressure, glucose, BMI, comorbidities) with statistical feature-selection methods (chi-square test, mutual information, recursive feature elimination) to keep only the most predictive features.
- Reducing to the most relevant features improves both KNN distance quality (irrelevant/noisy features distort distance in high dimensions) and RBF center placement (centers represent the true clusters of risk-relevant patient profiles rather than noise).

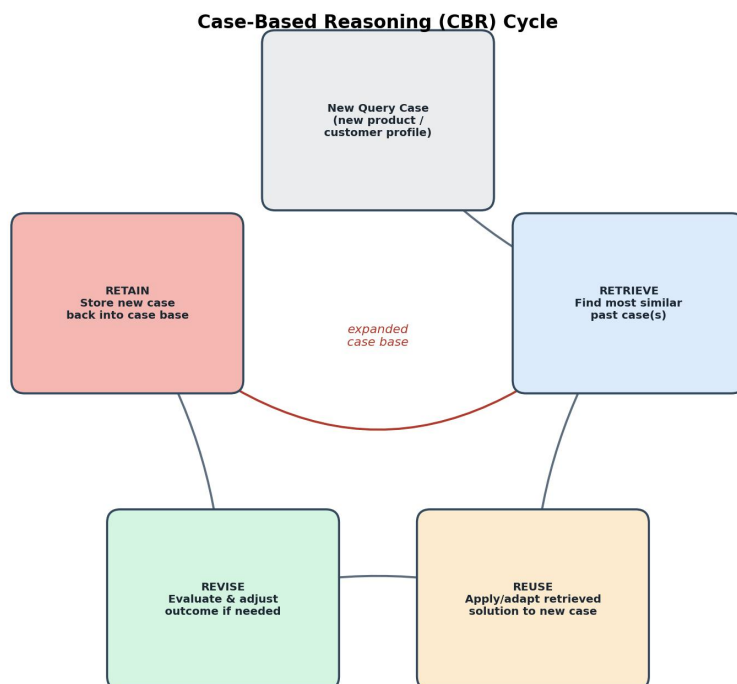
Improving Accuracy in Real-World Scenarios

- Feature scaling/normalization is essential for both methods, since raw clinical measurements have very different numeric ranges (e.g., glucose in mg/dL vs age in years).
- Use distance-weighted voting in KNN (closer neighbours count more) rather than a flat majority vote.
- Address class imbalance (high-risk patients are typically a minority) via weighted voting, SMOTE-style oversampling, or class-weighted RBF training, so the model doesn't default to predicting the majority (low-risk) class.
- Continuously validate with cross-validation as the patient population and clinical practices evolve, and pair the model's output with an expert-review workflow for flagged high-risk cases, given the high cost of a missed diagnosis.

4. Similar Product Detection for Dynamic Pricing (Case-Based Learning)

Proposed Design

Represent every product as a case: a feature vector (category, brand, specifications, price history, demand elasticity) plus its pricing outcomes (how sales responded to past price changes). When setting a price for a product — especially a new or low-history one — retrieve the most similar existing product cases and reuse their pricing/demand-response pattern to inform the new price, following the classic Case-Based Reasoning cycle:



Defining Similarity Metrics

- Categorical similarity: exact match or ontology/category-tree-based similarity for product category and brand.
- Numeric distance: weighted Euclidean/Manhattan distance over specifications and price range.
- Text/semantic similarity: cosine similarity over TF-IDF or embedding vectors of product titles/descriptions, useful for catching similar products that don't share identical structured attributes.
- These are combined into one composite similarity score via a weighted sum, with weights reflecting which attributes matter most for pricing decisions in that category (e.g., brand may matter more for electronics, specifications more for raw materials).

Reusing Past Cases

- Retrieve: find the top-K most similar products and their historical price–demand response.
- Reuse: adapt the retrieved pricing strategy to the new product's context (e.g., scale the suggested price change proportionally to the price-range difference between the retrieved product and the new one).
- Revise: monitor the actual sales outcome after applying the adapted price and adjust if the response differs from what was expected.
- Retain: store the new product and its observed outcome back into the case base, so future pricing decisions benefit from this additional evidence — this is what allows the system to keep improving over time without manual retraining.

Scaling with Increasing Product Data

- Replace brute-force similarity comparison with approximate nearest-neighbour search (e.g., HNSW or other vector-index structures) as the catalogue grows into the millions of products.
- Use learned product embeddings (e.g., from an autoencoder or a pretrained product/text model) to compress the feature representation, making large-scale similarity search both faster and more semantically meaningful than raw structured features alone.

- Periodically perform case-base maintenance (case-base editing): prune redundant, near-duplicate, or stale cases so retrieval stays fast and the case base remains representative rather than growing unboundedly.

5. Delivery Time Forecasting (RBF Network or Locally Weighted Regression)

Proposed Design

- RBF Network approach: cluster historical routes/conditions (e.g., via K-Means on location, time-of-day, and traffic-level features) to obtain representative centers; each Gaussian hidden unit activates for routes/conditions similar to its center, and the network combines these activations to predict delivery time for a new route.
- LWR approach: for a new delivery request (its location, time, and current traffic features), fit a local linear regression using only the most similar historical deliveries, weighted by a kernel over the feature distance — giving a query-specific prediction rather than one fixed global formula.
- Either approach is a reasonable fit; RBF is preferable when a compact, reusable trained model is needed for fast repeated inference, while LWR is preferable when maximum local accuracy per query matters more than inference speed and the historical dataset is manageable in size.

Capturing Non-Linear Relationships

- Delivery time depends on interacting, non-linear factors — e.g., a given distance takes far longer during downtown rush hour than the same distance on an off-peak highway route — relationships a single linear model cannot represent.
- RBF's Gaussian basis functions naturally carve the feature space into localized regions (e.g., "downtown + rush hour" vs "highway + off-peak"), each contributing its own learned response, which in combination approximates a highly non-linear delivery-time surface.
- LWR achieves the same flexibility differently: by fitting a fresh local linear approximation around every query's specific location/time/traffic profile, it traces the true non-linear relationship without needing to specify its functional form in advance.

Improving Prediction Reliability Under Varying Conditions

- Incorporate real-time contextual features (live traffic feed, weather, day-of-week/holiday flags) as inputs so predictions reflect current, not just historical average, conditions.
- Use recency-weighted or sliding-window training data so the model adapts to recent traffic-pattern changes (new road construction, seasonal patterns) rather than relying on stale historical averages.
- Report a prediction interval (derived from the spread of nearby historical delivery times) rather than a single point estimate — giving customers a reliable time range (e.g., "20–30 minutes") that reflects genuine uncertainty under variable traffic conditions, rather than false precision.
- Periodically retrain/update RBF centers (or refresh the LWR reference dataset) as new routes, delivery zones, or traffic patterns emerge, so the model doesn't degrade as the logistics network evolves.